

DEVELOPER DOCUMENTATION

Laravel API Evolution

v5.8 → v12

A comprehensive reference guide covering every significant API change, new programming pattern, and authentication evolution across all major Laravel versions.

[Versions 5.8 – 12](#)[Authentication & Security](#)[Routing & Resources](#)[Best Practices](#)

[laravel.com/docs](#) · Compiled February 2026 · PHP 7.2 → PHP 8.2+

Overview: The API Journey

Laravel's API capabilities have evolved dramatically since version 5.8. From basic token guards and Passport-only OAuth, to the lightweight Sanctum, to the streamlined opt-in structure of Laravel 11/12 — each version brought meaningful improvements to how developers build and secure APIs.

- **v5.8 (Feb 2019) — SHA-256 Token Hashing**
Basic token guard updated to support hashed token storage for improved security.
- **v6.0 (Sep 2019) — Semantic Versioning, Vapor Support, Lazy Collections**
Framework aligned to semver; lazy collections improved large API dataset handling. First LTS after 5.x.
- **v7.0 (Mar 2020) — Sanctum, HTTP Client, Route Caching**
Laravel Sanctum debuted as a lightweight SPA/token auth alternative to Passport. New `Http::` facade for consuming external APIs.
- **v8.0 (Sep 2020) — Model Factories, Job Batching, Improved Rate Limiting**
Massive improvements to testing and background processing patterns for APIs. Dynamic rate limiting per user.
- **v9.0 (Feb 2022) — Symfony 6, Scoped Bindings, Enum Support**
Native PHP 8.1 enum support in routes and scoped nested resource bindings.
- **v10.0 (Feb 2023) — PHP 8.1 Minimum, Pennant Feature Flags**
Feature flags with Pennant for API rollouts; full PHP 8.1 native types throughout.
- **v11.0 (Mar 2024) — Slim Skeleton, Opt-in API, Per-Second Rate Limiting**
Biggest structural change: `api.php` removed by default, installed on demand. Centralized `bootstrap/app.php`.
- **v12.0 (Feb 2025) — Starter Kits, WorkOS AuthKit, Stability Release**
New React/Vue/Livewire starter kits with modern auth including passkeys and SSO support.

[v5.8] Laravel 5.8 — API Security Foundations

February 2019

■ SHA-256 Token Hashing

The built-in token guard now stores API tokens as SHA-256 hashes instead of plain text, significantly improving security for simple token-based APIs.

● Security Improvement

■ Improved Email Validation

Adopted the egulias/email-validator package, fixing edge cases in API input validation where valid international email addresses were rejected.

● Improved

■ Eloquent hasOneThrough

New relationship type allowing navigation through intermediate models in a single query — useful for deeply nested API resource relationships.

● New

■ Token Guard Still Basic

The simple token guard remained the only built-in lightweight option. Passport was the recommended solution for production APIs. Sanctum arrives in v7.0.

● Context

config/auth.php — SHA-256 Token Hashing (v5.8)

```
// Enable hashed token storage in your guard config
'api' => [
    'driver' => 'token',
    'provider' => 'users',
    'hash' => true, // NEW in v5.8 – stores SHA-256 hash
],
// When creating a token for a user:
$token = Str::random(60);
$user->forceFill([
    'api_token' => hash('sha256', $token),
])->save();
// Return the plain token to the client (hash is in the DB)
return response()->json(['token' => $token]);
```

Note: The built-in token guard was always meant for simple use cases. Laravel already recommended Passport (OAuth2) for production APIs in v5.8. Sanctum would arrive in v7.0 as the lightweight alternative.

[v6.0 LTS] Laravel 6.0 — Lazy Collections, Vapor & Policy Responses

September
2019

■ Lazy Collections

New LazyCollection class using PHP generators to stream large database result sets through API responses without loading everything into memory at once.

● New

■ Laravel Vapor Compatibility

Full compatibility with serverless deployment on AWS Lambda, making scalable API infrastructure dramatically simpler with SQS, Redis, and CloudFront support.

● New

■ Authorization Policy Responses

Policies can now return detailed error messages via Response::deny(), giving API clients meaningful 403 responses rather than generic Forbidden messages.

● Improved

■ Semantic Versioning

laravel/framework now follows semver. API dependency upgrades are now predictable — you know exactly when breaking changes occur.

● Changed

App\Policies\PostPolicy.php — Richer API Auth Responses (v6.0)

```
use Illuminate\Auth\Access\Response;

public function update(User $user, Post $post): Response
{
    return $user->id === $post->user_id
    ? Response::allow()
    : Response::deny('You do not own this post.');
```

// API clients now receive:
// {"message": "You do not own this post."} with HTTP 403

```
}
```

Api/ExportController.php — Lazy Collections for large datasets (v6.0)

```
// Stream 100,000+ records without memory explosion
public function export()
{
    return response()->streamDownload(function () {
        // cursor() returns a LazyCollection – one record at a time
        Post::cursor()->each(function ($post) {
            echo json_encode($post) . "\n";
        });
    }, 'posts.jsonl');
```

[v7.0] Laravel 7.0 — Sanctum Born & HTTP Client Arrives

March 2020

Major API Release: Laravel 7 introduced Sanctum — a lightweight authentication package that became the go-to solution for SPA and mobile API authentication, replacing the need for full OAuth2 via Passport in most cases.

■■ Laravel Sanctum

A featherweight authentication system for SPAs, mobile apps, and token-based APIs. Supports cookie-based session auth AND personal API tokens without OAuth2 overhead.

● New Package

■ HTTP Client (Http:: Facade)

New first-party Http facade wrapping Guzzle. Consuming external APIs from Laravel is now clean and testable with built-in fake/mock support — no more raw Guzzle boilerplate.

● New

■ Fluent String Operations

New Str::of() object-oriented string manipulation, useful for transforming API response data in a clean, chainable way.

● New

■ Custom Route Model Binding

Custom resolution logic for route model bindings, allowing APIs to resolve models by any attribute (slug, uuid, etc.), not just the primary key id.

● Improved

routes/api.php — Sanctum Authentication Setup (v7.0)

```
// 1. Install: composer require laravel/sanctum
// 2. Add HasApiTokens trait to User model
use Laravel\Sanctum\HasApiTokens;
class User extends Authenticatable
{
    use HasApiTokens, HasFactory, Notifiable;
}

// 3. Issue a scoped token on login
Route::post('/login', function (Request $request) {
    // ... validate credentials ...
    $token = $request->user()->createToken(
        'mobile-app', ['posts:read', 'posts:write']
    );
    return response()->json(['token' => $token->plainTextToken]);
});

// 4. Protect routes with Sanctum middleware
Route::middleware('auth:sanctum')->group(function () {
    Route::get('/user', fn (Request $r) => $r->user());
    Route::apiResource('posts', PostController::class);
});

// 5. Check token abilities in controllers
if (! $request->user()->tokenCan('posts:write')) {
    return response()->json(['error' => 'Insufficient scope'], 403);
}
```

App\Services — New HTTP Client for external APIs (v7.0)

```
use Illuminate\Support\Facades\Http;
// Clean, fluent, fully testable API consumption
$response = Http::withToken($apiToken)
->retry(3, 100) // Retry 3 times, 100ms delay
->timeout(10) // 10 second timeout
->acceptJson()
->get('https://api.example.com/users', [
    'page' => 1,
    'limit' => 50,
]);
if ($response->successful()) {
    return $response->json();
}
$response->throw(); // Throws HttpClientException on error
// In feature tests:
Http::fake(['api.example.com/*' => Http::response(['data' => []])]);
```

[v8.0] Laravel 8.0 — Factories, Job Batching & Dynamic Rate Limiting

September
2020

■ Class-Based Model Factories

Factories moved from functions to dedicated classes with HasFactory trait. Makes API testing far more expressive with factory states, sequences, and relationships.

● Redesigned

■ Job Batching

Bus::batch() allows dispatching groups of API-processing jobs and tracking collective completion, progress, and failure with callbacks.

● New

■ Dynamic Rate Limiting

Rate limits dynamically computed per user. Premium users get higher limits, free users get lower — all configured elegantly in RouteServiceProvider.

● Improved

■ Route Controller Groups

New Route::controller() groups routes by controller class, dramatically reducing repetition in large api.php files.

● New

■ Invokable Validation Rules

Custom API validation rules can now be simple invokable classes instead of requiring separate passes() and message() methods.

● New

■ Maintenance Mode Bypass Tokens

Maintenance mode supports bypass tokens so API clients with a secret key can continue operating during maintenance windows.

● Improved

App\Providers\RouteServiceProvider.php — Dynamic Rate Limiting (v8.0)

```
use Illuminate\Cache\RateLimiting\Limit;
use Illuminate\Support\Facades\RateLimiter;

protected function configureRateLimiting(): void
{
    RateLimiter::for('api', function (Request $request) {
        // Dynamic limit based on user subscription plan
        return Limit::perMinute(
            $request->user()?->is_premium ? 1000 : 60
        )->by($request->user()?->id ?: $request->ip());
    });
}

// Apply in routes/api.php:
Route::middleware(['auth:sanctum', 'throttle:api'])->group(function () {
    Route::apiResource('posts', PostController::class);
});
```

routes/api.php — Route Controller Grouping (v8.0)

```
// Before v8.0 – repetitive controller class references
Route::get('/posts', [PostController::class, 'index']);
Route::post('/posts', [PostController::class, 'store']);
Route::get('/posts/{post}', [PostController::class, 'show']);
// v8.0+ – cleaner grouping by controller
Route::controller(PostController::class)->group(function () {
Route::get('/posts', 'index');
Route::post('/posts', 'store');
Route::get('/posts/{post}', 'show');
Route::put('/posts/{post}', 'update');
Route::delete('/posts/{post}', 'destroy');
});
```


[v9.0] Laravel 9.0 — Enums, Scoped Bindings & Symfony 6

February 2022

■ PHP 8.1 Enum Route Binding

Route model binding natively supports PHP 8.1 backed enums. Pass an enum value in the URL and Laravel resolves it automatically — invalid values return 404.

● New

■ Scoped Route Model Binding

Automatically scope nested resource routes. `/posts/{post}/comments/{comment}` ensures the comment belongs to the post — no manual ownership checks needed.

● New

■ Symfony Mailer Upgrade

Replaced SwiftMailer with Symfony Mailer, bringing modern email transport support. Affects APIs that trigger transactional email notifications.

● Breaking Change

■ Enum Casting in Models

Eloquent models can now cast attributes to PHP enums. API responses automatically serialize enum values to their string or integer representations.

● New

routes/api.php — Enum Route Binding (v9.0)

```
// Define a PHP 8.1 backed enum
enum Status: string
{
    case Active = 'active';
    case Inactive = 'inactive';
    case Banned = 'banned';
}

// Route with enum binding – invalid values auto-return 404
Route::get('/users/status/{status}', function (Status $status) {
    return User::where('status', $status->value)->paginate();
});

// GET /api/users/status/active -> 200 OK with users
// GET /api/users/status/unknown -> 404 Not Found (automatic)
```

routes/api.php — Scoped Route Bindings (v9.0)

```
// Ensures {comment} belongs to {post} – no manual check needed
Route::scopeBindings()->group(function () {
    Route::get('/posts/{post}/comments/{comment}', [
        CommentController::class, 'show'
    ]);
    Route::put('/posts/{post}/comments/{comment}', [
        CommentController::class, 'update'
    ]);
});

// GET /posts/5/comments/99
// -> 404 if comment 99 doesn't belong to post 5
// -> 200 if it does – ownership enforced automatically
```

[v10.0] Laravel 10.0 — Pennant Feature Flags & Native Types February 2023

■ Laravel Pennant (Feature Flags)

First-party feature flag package. Roll out new API endpoints to a percentage of users or based on custom logic — without deploying separate code branches.

● New Package

■ Native PHP Type Declarations

All Laravel framework code now uses native PHP type declarations. Better IDE autocompletion, static analysis support, and safer API controller development.

● Changed

■■ Process Facade

New Process facade for running shell commands — useful in API workflows that trigger CLI tools or external processes, with full fake/test support.

● New

■ Test Profiling

php artisan test --profile reveals the slowest tests. Essential for keeping large API integration test suites fast as they grow.

● New

App\Providers\AppServiceProvider.php — Pennant Feature Flags (v10.0)

```
use Laravel\Pennant\Feature;

public function boot(): void
{
    // 20% rollout of new recommendations engine
    Feature::define('new-recommendations-engine', fn (User $user) =>
        $user->is_beta_tester || random_int(1, 100) <= 20
    );
    // Always active for premium users
    Feature::define('advanced-analytics', fn (User $user) =>
        $user->subscription === 'premium'
    );
}

// In an API controller:
public function recommendations(Request $request): JsonResponse
{
    if (Feature::active('new-recommendations-engine')) {
        return response()->json(
            NewRecommendationService::for($request->user())
        );
    }
    return response()->json(
        LegacyRecommendationService::for($request->user())
    );
}
```

[v11.0] Laravel 11.0 — The Slim API Structure Revolution

March 2024

Biggest API Change Yet: Laravel 11 completely reimaged how API apps are structured. `routes/api.php` no longer exists by default — it is created on demand via `artisan install:api`. Sanctum is also opt-in. This is the most significant structural API change since Laravel 5.

■ Opt-In API Routing

`routes/api.php` is gone from new projects. Run `php artisan install:api` to scaffold it along with Sanctum configuration automatically.

● New Projects

■ Per-Second Rate Limiting

Rate limiters now support per-second granularity with `perSecond()` — critical for high-throughput API endpoints that need burst protection.

● New

■ Revamped `bootstrap/app.php`

All routing, middleware, exception handling, and service providers are now configured in a single file, replacing the scattered config of previous versions.

● Redesigned

■ Health Check Endpoint `/up`

Built-in GET `/up` route for monitoring services, load balancer health checks, and uptime monitoring — no custom route or controller needed.

● New

■ Encryption Key Rotation

Define previous encryption keys alongside the current key, allowing zero-downtime key rotation for encrypted API tokens and cookies.

● New

■ `once()` Memoization Helper

New `once()` helper memoizes the result of a closure for the lifetime of a request — ideal for expensive operations called multiple times in an API response pipeline.

● New

terminal — Setting up API in a new Laravel 11 project

```
# Create new Laravel 11 project
composer create-project laravel/laravel my-api
# Install API scaffolding (all-in-one command)
php artisan install:api
# This single command:
# - Creates routes/api.php
# - Registers api.php in bootstrap/app.php
# - Installs and configures Laravel Sanctum
# - Runs Sanctum migrations (personal_access_tokens table)
# - Reminds you to add HasApiTokens to your User model
# Install broadcasting separately if needed
php artisan install:broadcasting
```

bootstrap/app.php — New centralized app configuration (v11.0)

```
use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        api: __DIR__.'/../routes/api.php', // Added by install:api
        health: '/up', // Built-in health endpoint
    )
    ->withMiddleware(function (Middleware $middleware) {
        // Append custom middleware to the API group
        $middleware->api(append: [
            EnsureApiVersion::class,
            ForceJsonResponse::class,
        ]);
    })
    ->withExceptions(function (Exceptions $exceptions) {
        // Return JSON for all API routes automatically
        $exceptions->shouldRenderJsonWhen(
            fn (Request $r) => $r->is('api/*') || $r->wantsJson()
        );
    })
    ->create();
```

App\Providers\AppServiceProvider.php — Per-Second Rate Limiting (v11.0)

```
public function boot(): void
{
    RateLimiter::for('api', function (Request $request) {
        return [
            Limit::perSecond(10), // Burst protection – NEW in v11
            Limit::perMinute(200) // Sustained throughput limit
        ]->by($request->user()?->id ?: $request->ip()),
    ];
};
```

[v12.0] Laravel 12.0 — Modern Starter Kits & WorkOS Auth

February 2025

Stability-Focused Release: Laravel 12 is intentionally a low-breaking-change release. Most applications upgrade with zero code changes. The focus was on dependency upgrades and shipping new modern starter kits to replace Breeze and Jetstream.

■ New Starter Kits (React, Vue, Livewire)

Completely rebuilt starter kits for React, Vue, and Livewire. Each ships with full authentication scaffolding and API integration, replacing Breeze and Jetstream.

● New

■ WorkOS AuthKit Integration

Optional WorkOS variant provides social auth, passkeys, and enterprise SSO — major upgrade for APIs requiring enterprise-grade authentication without building it from scratch.

● New

■ Breeze & Jetstream Maintenance Mode

Breeze and Jetstream enter maintenance-only status. All new feature development moved to the new starter kits. Existing projects continue to work normally.

● Changed

Authentication: Choosing the Right Tool

Laravel's authentication landscape evolved from a single simple token guard to three distinct solutions, each with a clear purpose. Understanding which to use is one of the most common questions when building a new Laravel API.

Feature / Scenario	Token Guard (v5.x)	Laravel Passport	Laravel Sanctum
Introduced in	Laravel 5.2	Laravel 5.3	Laravel 7.0
OAuth2 Support	✗	Full OAuth2	✗
SPA Authentication	✗	Partial	Yes (cookie-based)
Mobile App Tokens	Basic only	Yes	Yes (recommended)
Token Scopes/Abilities	✗	Full scopes	Abilities
Token Revocation	✗	Yes	Yes
Refresh Tokens	✗	Yes	✗
Third-Party API Access	✗	Yes (via OAuth)	Not intended
Setup Complexity	Very Simple	Complex (5+ DB tables)	Simple (1 DB table)
Best For	Simple internal tools	OAuth2 & enterprise APIs	SPAs, mobile, first-party

Decision Guide — Which authentication to use?

Does your API need THIRD-PARTY apps to act on behalf of users?
YES -> Use Laravel Passport (OAuth2 Authorization Code Grant)
NO |
Is it a first-party SPA (Vue, React, Next.js) on same/subdomain?
YES -> Use Laravel Sanctum (cookie-based session auth + CSRF)
NO |
Is it a mobile app or API consumer needing long-lived tokens?
YES -> Use Laravel Sanctum (personal access tokens)
NO |
Simple internal tool with low security requirements?
YES -> Token Guard works, but consider Sanctum anyway

Modern API Best Practices (Laravel 11+)

A reference for building production-grade APIs incorporating all patterns introduced across versions 5.8 through 12.

[App\Http\Resources\PostResource.php](#) — API Resources

```
namespace App\Http\Resources;
use Illuminate\Http\Resources\Json\JsonResource;
class PostResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'title' => $this->title,
            'body' => $this->body,
            'status' => $this->status->value, // Enum (v9+)
            'author' => new UserResource($this->whenLoaded('user')),
            'tags' => TagResource::collection($this->whenLoaded('tags')),
            'created_at' => $this->created_at->toISOString(),
            // Conditionally include admin-only fields
            $this->mergeWhen($request->user()?->is_admin, [
                'internal_notes' => $this->internal_notes,
            ]),
        ];
    }
}
```

[App\Http\Controllers\Api\PostController.php](#) — Complete Modern Controller

```

class PostController extends Controller
{
    public function index(): AnonymousResourceCollection
    {
        $posts = Post::with(['user', 'tags']) // Eager load – avoid N+1
        ->latest()
        ->paginate(15); // Always paginate lists
        return PostResource::collection($posts);
    }

    public function store(StorePostRequest $request): PostResource
    {
        // Form Request handles all validation
        $post = $request->user()->posts()->create($request->validated());
        return (new PostResource($post))
        ->response()->setStatusCode(201); // 201 Created
    }

    public function show(Post $post): PostResource // Route model binding
    {
        return new PostResource($post->load(['user', 'tags']));
    }

    public function destroy(Post $post): Response
    {
        $this->authorize('delete', $post); // Policy check
        $post->delete();
        return response()->noContent(); // 204 No Content
    }
}

```

routes/api.php — API Versioning Best Practice

```

// Version 1 API
Route::prefix('v1')->name('v1.')->group(function () {
    Route::middleware('auth:sanctum')->group(function () {
        Route::apiResource('posts', V1\PostController::class);
        Route::apiResource('comments', V1\CommentController::class);
    });
    Route::get('posts', [V1\PostController::class, 'index']); // Public
});

// Version 2 API (with new features)
Route::prefix('v2')->name('v2.')->group(function () {
    Route::middleware('auth:sanctum')->group(function () {
        Route::apiResource('posts', V2\PostController::class);
        Route::apiResource('reactions', V2\ReactionController::class);
    });
});

```

Quick Reference: Feature Availability by Version

Feature	v5.8	v6	v7	v8	v9	v10	v11	v12
API Resources	✓	✓	✓	✓	✓	✓	✓	✓
Laravel Sanctum	✗	✗	✓	✓	✓	✓	✓	✓
Http:: Client Facade	✗	✗	✓	✓	✓	✓	✓	✓
Dynamic Rate Limiting	✗	✗	✗	✓	✓	✓	✓	✓
Enum Route Binding	✗	✗	✗	✗	✓	✓	✓	✓
Scoped Bindings	✗	✗	✗	✗	✓	✓	✓	✓
Pennant (Feature Flags)	✗	✗	✗	✗	✗	✓	✓	✓
Per-Second Rate Limit	✗	✗	✗	✗	✗	✗	✓	✓
Opt-In API Routing	✗	✗	✗	✗	✗	✗	✓	✓
WorkOS / Passkey Auth	✗	✗	✗	✗	✗	✗	✗	✓